Brew Guide

Building Modern Web Apps with Jaspr, Gemini and Antigravity

Flutter Con | July 16, 2026 | Prompts, Commands, and Decisions

New in the development world? Don't you worry we have got you covered.

This document captures the exact workflow used to build Brew Guide using Spec-Driven Development. Every terminal command, every agy prompt, every spec file, and every decision is documented in the order it was executed. This is not a code reference, it is a process reference.

1. What is Spec-Driven Development

Spec-Driven Development (SDD) is a methodology where you write a structured specification document before writing any code. The spec describes what a component should do, what HTML it should render, what inputs it accepts, and what constraints it must follow. That spec becomes the input for an AI agent (agy) which generates the code.

The developer's role in SDD is not to write syntax. It is to design behavior. This is especially powerful when the developer has deep domain knowledge (specialty coffee, in this case) but is newer to a specific framework (Jaspr).

The SDD loop

1. Write a spec in plain language describing the component.
2. Give the spec to agy via the Agent panel or terminal.
3. Review the generated code against the spec.
4. Correct any issues and update the spec if needed.
5. Run dart analyze to confirm zero errors.
6. Test in the browser.

SDD PRINCIPLE

The spec is more valuable than the generated code. If agy fails or generates incorrect output, the spec lets you try again or explain the component manually to the audience. Always keep specs updated as decisions change.

2. Phase 1 — Environment setup

Run these commands once before starting the project. Verify each output before continuing.

2.1 Verify base tools

TERMINAL — verify Flutter and Dart

```
flutter --version
# Expected: Flutter 3.44.1, Dart 3.12.1

dart --version
# Expected: Dart SDK version 3.12.1

which dart
# Expected: /Users/aiarguelles/Development/flutter/bin/dart
```

2.2 Install Jaspr CLI

TERMINAL — install Jaspr CLI

```
dart pub global activate jaspr_cli

# Add to PATH if not already present:
export PATH="$PATH":"$HOME/.pub-cache/bin"

jaspr --version
# Expected: 0.23.1
```

2.3 Install and verify Antigravity CLI

TERMINAL — install and authenticate agy

```
# Install Antigravity CLI
curl -fsSL https://antigravity.google/cli/install.sh | bash

# Open a new terminal after install, then verify:
agy --version
# Expected: 1.0.5

# Authenticate with Google account:
agy auth
# Opens browser — sign in with your Google account
```

2.4 Verify Node.js

TERMINAL — verify Node

```
node --version    # Expected: v26.0.0
npm --version     # Expected: 11.12.1
```

2.5 Get Gemini API key

Go to aistudio.google.com. Sign in with your personal Google account. The GCP project associated with the key must have billing enabled. Free tier quota is zero for all models, every request will fail with 429 without billing.

TERMINAL — verify Gemini key

```
curl
"https://generativelanguage.googleapis.com/v1beta/models/gemini-3.5-flash:generate
Content" \
  -H "Content-Type: application/json" \
  -H "X-goog-api-key: YOUR_KEY_HERE" \
  -X POST \
  -d '{"contents":[{"parts":[{"text":"say hello"}]}]}'

# Expected: JSON response with candidates array
# If you see 403: wrong key or no billing
# If you see 429: quota exhausted, check billing
```

3. Phase 2 — Create the Jaspr project

TERMINAL

```
cd /Users/aiarguelles/Development
jaspr create flutter-conf-jaspr-demo
```

Answer the CLI prompts as follows. Each answer has a reason.

Specify a target directory

TYPE THIS

```
flutter-conf-jaspr-demo
```

This becomes the Dart package name: flutter_conf_jaspr_demo (underscores).

Select a rendering mode

SELECT THIS

```
server  (use arrow keys, select server, press Enter)
```

Server mode runs a Dart HTTP server that generates HTML per request. Required for SSR and for the Gemini integration. Static generates HTML once at build time. Client is equivalent to Flutter Web.

Setup routing for different pages? (Y/n)

TYPE THIS

```
Y
```

Brew Guide has two pages: the method selector at / and the coffee detail at /coffee.

Prompt: Use multi-page (server-side) routing? (Y/n)

```
TYPE THIS
Y
```

Each route renders on the server independently. Better for SSR demonstration in DevTools.

Setup web embedding? (y/N)

```
TYPE THIS
N
```

Web embedding is for projects that mix Flutter Web and Jaspr. Brew Guide is Jaspr only.

Enable support for Flutter web plugins? (y/N)

```
TYPE THIS
N
```

No Flutter component exists in this project. No Flutter plugins needed.

> **VERIFY**
> Expected output: Generated 28 file(s). After this, run: cd flutter-conf-jaspr-demo && jaspr serve to verify the scaffold runs at localhost:8080.

---

4. Phase 3 — Fix the scaffold before building

The generated scaffold has problems that must be fixed before any other work. These are not optional.

4.1 Fix pubspec.yaml — change mode to server

Open pubspec.yaml. Find the jaspr section at the bottom. Change mode from static to server.

pubspec.yaml — find and change this

```
# WRONG (default)
jaspr:
  mode: static

# CORRECT
jaspr:
  mode: server
```

CRITICAL — do this first

Without this change, jaspr serve reports "Running jaspr in static rendering mode" even though you selected server during jaspr create. The Gemini handler will not work in static mode. This is the most critical fix in the entire project.

4.2 Add shelf dependencies to pubspec.yaml

The server middleware requires shelf and shelf_router. Add these under dependencies before running dart pub get.

pubspec.yaml — complete dependencies section

```
dependencies:
  jaspr: ^0.23.1
  jaspr_router: ^0.8.2
  http: ^1.2.0
  shelf: ^1.4.0         # ADD THIS
  shelf_router: ^1.1.0  # ADD THIS

dev_dependencies:
  build_runner: ^2.10.0
  build_web_compilers: ^4.8.0
  jaspr_builder: ^0.23.1
  lints: ^5.0.0

jaspr:
  mode: server
```

TERMINAL — install dependencies

```
dart pub get
# Expected: Changed 2 dependencies! (shelf, shelf_router added)
```

4.3 Fix main.server.dart — remove hardcoded styles and add middleware

The scaffolded main.server.dart has hardcoded CSS that conflicts with theme.dart. Replace the entire file. This version removes the style conflict and registers the /api/gemini POST route using ServerApp.addMiddleware.

lib/main.server.dart — replace entire file

```
library;

import 'package:jaspr/server.dart';
import 'app.dart';
import 'handlers/gemini_handler.dart';
import 'main.server.options.dart';

void main() {
  Jaspr.initializeApp(options: defaultServerOptions);

  ServerApp.addMiddleware((handler) {
    return (request) async {
      if (request.method == 'POST' &&
          request.url.path == 'api/gemini') {  // NO leading slash
        return handleGeminiRequest(request);
      }
      return handler(request);
    };
  });

  runApp(Document(
    title: 'Brew Guide',
    styles: [],
    body: App(),
  ));
}
```

4.4 Create the API key files

TERMINAL — create API key files

```
# Create .env with your actual key (no space after =)
echo "GEMINI_API_KEY=your_actual_key_here" > .env

# Add to .gitignore
echo ".env" >> .gitignore
echo "start.sh" >> .gitignore

# Create start.sh
echo '#!/bin/bash' > start.sh
echo 'export GEMINI_API_KEY="your_actual_key_here"' >> start.sh
echo 'jaspr serve' >> start.sh
chmod +x start.sh
```

ALWAYS USE ./start.sh

Never use jaspr serve directly after this point. Always use ./start.sh. The GEMINI_API_KEY environment variable is lost when the terminal closes. ./start.sh re-exports it every time.

4.5 Create required directories

TERMINAL — create directories

```
mkdir -p lib/handlers
mkdir -p lib/services
mkdir -p lib/specs
```

---

5. Phase 4 — Write the specs before using agy

Before running any agy prompt, write the spec files. This is the core of Spec-Driven Development. The specs tell agy exactly what to build.

5.1 Do you write the spec, or does agy write it?

Both options are valid. Which one you choose depends on the context and is a decision you can make consciously.

Option A: You write the spec manually

You describe the component in structured markdown. You decide the HTML structure, the constraints, the color tokens to use. Then you give that spec to agy to generate the code.

- Best when: you have a clear vision of what the component should look like.
- Best when: you want full control over the HTML structure.
- Best when: you are learning the spec format and want to practice.

Option B: agy writes the spec, then agy writes the code

You describe what you want in natural language. agy generates the spec. You review and correct the spec. Then agy generates the code from that spec. This is a two-step AI workflow with two layers of human review.

- Best when: you want to move faster.
- Best when: the component is complex and you want agy to propose the structure first.
- Best when: you want to demonstrate SDD on stage with maximum impact, the audience sees agy working twice.

DEMO RECOMMENDATION — use Option B on stage

For the demo on stage, Option B is more powerful. You say: "I describe what I want. agy writes the spec. I review the spec. agy writes the code from the spec. I review the code." The audience sees two layers of AI assistance and two layers of human judgment. That is the SDD message.

Option B prompt — ask agy to write the spec

AGY PROMPT — generate spec (Option B)

```
Create lib/specs/method_selector.md.
This spec describes a Jaspr StatelessComponent that displays
four brewing method cards (pour over, espresso, cold brew, french press)
and a Gemini text input field.
Each card submits a POST to /api/gemini with a pre-defined prompt.
The component uses the SCA color palette from lib/constants/theme.dart.
Include HTML structure, color tokens, and constraints.
Output only the spec file in markdown format.
```

After agy generates the spec, review it carefully. Correct any details that do not match your vision. Then use the spec to generate the component with the prompts in Section 6.

HOW SPECS WERE BUILT IN THIS PROJECT

In this project, the specs in Section 5.3 were generated with agy assistance and then reviewed and corrected by hand. The constraints section in particular required human judgment: agy did not know about the @css placement rule, the raw CSS workarounds, or the onclick JS pattern until those were added manually to the spec.

Option B prompts — generate all three specs with agy

If using Option B, run these three prompts in order. Review and correct each spec before continuing to the next.

AGY SPEC PROMPT 1 — lib/specs/method_selector.md

```
Create the file lib/specs/method_selector.md.
This spec describes a Jaspr StatelessComponent called MethodSelector.
It displays four brewing method cards (Pour over, Espresso, Cold brew, French press)
and a Gemini text input field.
Each card is a form that submits POST to /api/gemini with a pre-defined prompt.
The text input also submits POST to /api/gemini.
There is a loading overlay (div#loading) shown via JavaScript onclick.
There is an optional error banner shown when hasError constructor param is true.
The component uses the SCA color palette from lib/constants/theme.dart.
Include: purpose, HTML structure, color token references, and constraints.
Constraints must include: StatelessComponent, @css on the class not State,
loading overlay hidden by default, hasError bool parameter.
Output only the markdown spec file.
```

AGY SPEC PROMPT 2 — lib/specs/coffee_detail.md

```
Create the file lib/specs/coffee_detail.md.
This spec describes a Jaspr component called Coffee in lib/pages/coffee.dart.
It displays a single coffee recommendation from Gemini.
It receives all data as constructor arguments:
name, origin, roast, method, description, brewTime, waterTemp, grind, flavorNotes.
flavorNotes is a List<String>.
It has a back link to / at the top.
The HTML structure uses: article as root, h1 for name,
dl/dt/dd for brew parameters, ul/li for flavor notes.
The component uses the SCA color palette from lib/constants/theme.dart.
Include: purpose, constructor arguments, HTML structure, color token references.
Constraints must include: @client annotation, no RouteState.of(context),
article as root not div, dl for metrics not divs, one return statement only.
Output only the markdown spec file.
```

AGY SPEC PROMPT 3 — lib/specs/gemini_service.md

```
Create the file lib/specs/gemini_service.md.
This spec describes a Dart server-side service in lib/services/gemini_service.dart.
It calls the Gemini API and returns a CoffeeBean record.
The CoffeeBean record has: name, origin, roast, method, description,
brewTime, waterTemp, grind, flavorNotes (List<String> of 4 items).
```

API endpoint:
https://generativelanguage.googleapis.com/v1beta/models/gemini-3.5-flash:generateContent
Authentication: X-goog-api-key header — never as a query parameter.
API key comes from Platform.environment["GEMINI_API_KEY"].
generationConfig must include maxOutputTokens: 2048 and responseMimeType: "application/json".
System prompt must constrain all field lengths to prevent JSON truncation.
Include: purpose, CoffeeBean record, API details, system prompt, constraints.
Output only the markdown spec file.

5.2 The workflow in four steps

- Write a spec file in lib/specs/ describing what the component must do.
- Give the spec to agy via the Agent panel in Antigravity IDE or via the terminal.
- Review the generated file: check that it matches the spec, run dart analyze, verify there are no extra files appended.
- Accept the file in the IDE review panel, or copy it to the correct location if using terminal.

5.3 The spec file format

SPEC FILE FORMAT — template

```
# Spec: ComponentName

Framework: Jaspr (Dart), version 0.23.1
Rendering: SSR | @client
File: lib/path/to/component.dart

## Purpose
One or two sentences describing what the component does.

## Inputs
List of constructor parameters or data sources.

## HTML structure
<element class="name">
  <child>content</child>
</element>

## Colors (from lib/constants/theme.dart)
Constant name: hex value — usage

## Constraints
- Use semantic HTML.
- CSS via @css static getter.
- No JavaScript inline.
- Output only the Dart file.
```

5.4 Create spec files via terminal

Run each cat command to create the spec files. These are the exact specs used to build Brew Guide.

5.4.1 method_selector.md

TERMINAL — create method_selector.md

```
cat > lib/specs/method_selector.md << 'EOF'
# Spec: MethodSelector

Framework: Jaspr (Dart), version 0.23.1
Rendering: SSR, no @client annotation
File: lib/components/method_selector.dart

## Purpose
Display four brewing method cards and a Gemini text input.
When a method card is clicked, POST to /api/gemini with a pre-defined prompt.
When the Gemini input is submitted, POST to /api/gemini with the user text.

## Methods
- pour_over: label "Pour over", description "Clean, floral, bright"
- espresso:  label "Espresso",  description "Bold, intense, crema"
- cold_brew: label "Cold brew", description "Smooth, low acid"
- french_press: label "French press", description "Full body, rich"

## HTML structure
<section class="method-selector">
  <h1>Brew Guide</h1>
  <p class="subtitle">Select a brewing method</p>
  <div class="method-grid">
    <form method="post" action="/api/gemini">
      <input type="hidden" name="preference" value="Pour over prompt" />
      <button type="submit" class="method-card">...</button>
    </form>
  </div>
  <form class="gemini-input" method="post" action="/api/gemini">
    <input type="text" name="preference" placeholder="Tell us a coffee to
explore..." />
    <button type="submit">Ask Gemini</button>
  </form>
  <div class="loading-overlay" id="loading">
    <span class="cup">☕</span>
    <p>Brewing your recommendation...</p>
  </div>
</section>

## Colors (from lib/constants/theme.dart)
Background: colorBackground #F5EFE4
Surface: colorSurface #FFFDF8
Border: colorBorder #E2D5BE
Active border: scaHerb #326359
Button background: scaWine #692729
Button text: scaCitrus #F6B36F

## Constraints
- StatelessComponent, no @client annotation.
- @css static getter must be on the MethodSelector class, not on any State class.
- CSS via @css static getter on the component class.
- Loading overlay hidden by default (display: none), shown via onclick JS.
```

```
- No JavaScript frameworks. Inline onclick only.
EOF
```

5.4.2 coffee_detail.md

```
TERMINAL — create coffee_detail.md
cat > lib/specs/coffee_detail.md << 'EOF'
# Spec: CoffeeDetail

Framework: Jaspr (Dart), version 0.23.1
Rendering: SSR
File: lib/pages/coffee.dart

## Purpose
Display a single coffee recommendation returned by the Gemini API.
Receives all data as constructor arguments (from URL query parameters).

## Constructor arguments
final String name;
final String origin;
final String roast;
final String method;
final String description;
final String brewTime;
final String waterTemp;
final String grind;
final List<String> flavorNotes;

## HTML structure
<div class="coffee-page">
  <a href="/" class="back-link">☕ Brew Guide</a>
  <article class="coffee-detail">
    <div class="tags">
      <span class="tag method">Pour over</span>
      <span class="tag roast">Light roast</span>
      <span class="tag origin">Ethiopia</span>
    </div>
    <h1>{name}</h1>
    <p class="description">{description}</p>
    <dl class="metrics">
      <dt>Grind</dt><dd>{grind}</dd>
      <dt>Water</dt><dd>{waterTemp}</dd>
      <dt>Time</dt><dd>{brewTime}</dd>
    </dl>
    <ul class="flavor-notes">
      <li>{note}</li>
    </ul>
  </article>
</div>

## Constraints
- Use <article> as root element, not <div>.
- Flavor notes use <ul> and <li>, not divs.
- @client annotation required (browser interactivity).
```

```
- CSS via @css static getter.
- Do NOT use RouteState.of(context) — read from constructor args only.
- build method must have only ONE return statement.
EOF
```

5.4.3 gemini_service.md — shown on stage during demo

```
TERMINAL — create gemini_service.md
cat > lib/specs/gemini_service.md << 'EOF'
# Spec: GeminiService

Framework: Dart server-side (not compiled to browser)
File: lib/services/gemini_service.dart

## Purpose
Call the Gemini API from the Dart server and return a structured
coffee recommendation based on a user preference string.

## CoffeeBean record
typedef CoffeeBean = ({
  String name,
  String origin,
  String roast,       // light | medium | dark
  String method,      // pour_over | espresso | cold_brew | french_press
  String description, // max 10 words
  String brewTime,
  String waterTemp,
  String grind,
  List<String> flavorNotes, // exactly 4 strings
});

## API details
- Endpoint:
https://generativelanguage.googleapis.com/v1beta/models/gemini-3.5-flash:generateC
ontent
- Authentication: X-goog-api-key header (never query parameter)
- API key: read from Platform.environment["GEMINI_API_KEY"]

## generationConfig
generationConfig: {
  responseMimeType: "application/json",
  maxOutputTokens: 2048,
}

## System prompt
You are a specialty coffee expert. Return ONLY a valid JSON object.
Use these exact fields and keep values SHORT:
name (3 words max), origin (1 word), roast (light|medium|dark),
method (pour_over|espresso|cold_brew|french_press),
description (10 words max), brewTime, waterTemp, grind (1 word),
flavorNotes (exactly 4 strings, 1-2 words each).
JSON only. No explanation. No markdown.

## Constraints
```

```
- maxOutputTokens: 2048 inside generationConfig, not as a root field
- responseMimeType: "application/json" inside generationConfig, not as a root
field
- Strip markdown backticks from response if present
- Return CoffeeBean record from parsed JSON
- All fields have null-safe fallbacks with ?? ""
EOF
```

---

6. Phase 5 — Generate files with agy

With the specs written, run these prompts in the Antigravity IDE Agent panel (right side panel) or in the terminal. The prompts are ordered by dependency: generate services before handlers, handlers before pages.

BEFORE RUNNING AGY

Before running any agy prompt: open Antigravity IDE with the project, grant all file system permissions when asked (select "Yes, and always allow"), and verify jaspr serve is NOT running in the IDE terminal to avoid port conflicts.

6.1 Generate theme.dart — SCA palette

Paste this prompt in the Agent panel. The SCA palette was designed for specialty coffee roasters and maps each color to a flavor category.

AGY PROMPT 1 — theme.dart

Update lib/constants/theme.dart with the SCA (Specialty Coffee Association)
flavor palette. Replace the entire file with:

- scaCitrus #F6B36F (maps to citrus flavor notes)
- scaStrawberry #FF5658 (strawberry/red fruit notes)
- scaChocolate #74301E (chocolate/cocoa notes)
- scaFloral #C28DBE (floral/jasmine notes)
- scaBlueberry #2659B1 (blueberry/dark fruit notes)
- scaFerment #CDAF29 (fermented/wine-like notes)
- scaWine #692729 (wine/grape notes)
- scaHerb #326359 (herbal/green notes)
- scaBay #143D37 (earthy/bay notes)
- Neutral palette: colorBackground #F5EFE4, colorSurface #FFFDF8,
  colorBorder #E2D5BE, colorTextDark #1C1208,
  colorTextMid #5C4A2A, colorTextMuted #8C7355

Import Playfair Display (weight 500, 700) and DM Sans (weight 400, 500, 700)
from Google Fonts. Set h1 to 3rem weight 700, p to 1rem weight 400.
Font weights must be 400 minimum for projector visibility.
Keep primaryColor = scaBlueberry as legacy alias.
Output only the Dart file.

AFTER ACCEPTING

After accepting: run dart analyze. If you see styles_ordering warnings, they are informational only and do not affect the build. There must be zero errors.

6.2 Generate header.dart

AGY PROMPT 2 — header.dart

Replace lib/components/header.dart with a simplified header.
The header shows only a "Brew Guide" wordmark that links to /.
No Home/About navigation — those routes no longer exist.

Use Playfair Display font, weight 500, colorTextDark color.
Add a bottom border using colorBorder.
Use jaspr_router Link component for the navigation.
Output only the Dart file.

6.3 Generate gemini_service.dart — THE DEMO MOMENT ON STAGE

Open lib/specs/gemini_service.md on screen first, explain the spec to the audience, then run this prompt.

AGY PROMPT 3 — gemini_service.dart (used on stage)

Generate lib/services/gemini_service.dart following
the spec in lib/specs/gemini_service.md exactly.
Output only the Dart file.

KNOWN AGY ISSUES — check these

REVIEW REQUIRED after accepting:
1. Check that the file ends at the closing } of the recommendCoffee function. agy sometimes appends the full contents of pubspec.yaml after the closing bracket. Delete everything after the final } of the function.
2. Verify generationConfig contains maxOutputTokens: 2048 and responseMimeType: "application/json" as nested fields, not as root-level fields.
3. Verify authentication uses X-goog-api-key header, not a query parameter in the URL.

6.4 Generate gemini_handler.dart

AGY PROMPT 4 — gemini_handler.dart

Create lib/handlers/gemini_handler.dart.
This is a server-side Dart file, NOT compiled to browser.
It must:
1. Import dart:io, package:jaspr/server.dart, ../services/gemini_service.dart
2. Define Future<Response> handleGeminiRequest(Request request)
3. Read GEMINI_API_KEY from Platform.environment
4. Read the "preference" field from the POST form body using Uri.splitQueryString
5. Call recommendCoffee(preference, apiKey)
6. On success: redirect to /coffee with all CoffeeBean fields as query parameters.
   Use bean.flavorNotes.join(",") for the flavorNotes field.
   Use path: "/coffee" without trailing slash in the Uri.
7. On error: redirect to /?error=1&t=${DateTime.now().millisecondsSinceEpoch}
Output only the Dart file.

CHECK THIS

REVIEW: Check that flavorNotes uses .join(",") not .jointoString(","). This is a known agy hallucination with List methods that compiles but produces wrong output at runtime.

6.5 Generate method_selector.dart

AGY PROMPT 5 — method_selector.dart

Generate lib/components/method_selector.dart following
the spec in lib/specs/method_selector.md exactly.
Additional requirements not in the spec:

- Each card is a <form> with a hidden input containing a pre-defined prompt:
  Pour over: "Pour over coffee recommendation, light roast, floral and clean"
  Espresso: "Espresso coffee recommendation, dark roast, bold and intense with crema"
  Cold brew: "Cold brew coffee recommendation, smooth and low acid, refreshing"
  French press: "French press coffee recommendation, medium dark roast, full body"

- Every submit button has this exact onclick attribute value (copy exactly):
  var url = new URL(window.location.href);
  url.searchParams.delete("error");
  url.searchParams.delete("t");
  history.replaceState({}, "", url.pathname);
  document.getElementById("loading").style.display="flex";
  window.addEventListener("pageshow", function(e) {
    if(e.persisted) { document.getElementById("loading").style.display="none"; }
  }, {once: true});

- If hasError is true, show a div.error-banner before the method-grid with:
  text "Something went wrong. Please try again."
  alignSelf: AlignSelf.center
  wine/red background color using const Color("#6927291a")
  color: scaWine

- The loading overlay uses CSS @keyframes spin animation on the cup span.
- .gemini-input uses raw: {"width": "100%", "box-sizing": "border-box"}
- Ask Gemini button uses raw: {"white-space": "nowrap"}
- .method-selector uses alignItems: AlignItems.stretch
- Component has a bool hasError = false constructor parameter.
- The @css static getter must be on the MethodSelector class directly,
  not on any State class.

Use only color constants from lib/constants/theme.dart.
Output only the Dart file.

CHECK THIS

REVIEW: Verify @css static getter is on class MethodSelector, not on MethodSelectorState. If it is on the State class, the build will fail with: The getter style is not defined for the type MethodSelector.

6.6 Generate coffee.dart (detail page)

AGY PROMPT 6 — coffee.dart

Generate lib/pages/coffee.dart following
the spec in lib/specs/coffee_detail.md exactly.
The component must have @client annotation.
All CoffeeBean fields are final constructor arguments, not read from RouteState.

Add a back link above the article: ☕ Brew Guide
Wrap everything in a div.coffee-page.
Do NOT use RouteState.of(context) — data comes from constructor args only.
Do NOT add extension methods on BuildContext — they cause compilation errors.
The build method must have exactly ONE return statement.
Use only color constants from lib/constants/theme.dart.
Output only the Dart file.

CHECK THIS

REVIEW: Check that the build method has only ONE return statement. agy sometimes generates the new code and leaves the old code below it with a second return statement. The second return and everything after it must be deleted.

6.7 Update home.dart

AGY PROMPT 7 — home.dart

Replace lib/pages/home.dart with a minimal component.
It must:
- Have @client annotation
- Accept a bool hasError = false constructor parameter
- Import and return MethodSelector(hasError: hasError)
- Nothing else.
Output only the Dart file.

6.8 Update app.dart

AGY PROMPT 8 — app.dart

Replace lib/app.dart router with two routes only:
1. Route path "/" → Home(hasError: state.queryParams["error"] == "1")
2. Route path "/coffee" → Coffee with all CoffeeBean fields from state.queryParams.
   flavorNotes: (state.queryParams["flavorNotes"] ?? "").split(",")
Remove the /about route entirely.
Keep the Header component and .main div wrapper.
Do not add trailing slash to /coffee — jaspr_router throws an assertion error.
Output only the Dart file.

7. Phase 6 — Verify and run

7.1 Run dart analyze

```
TERMINAL — must pass before ./start.sh
dart analyze
# Expected: only "info" warnings about styles_ordering
# Zero errors required before running the server
```

7.2 If ports are blocked

```
TERMINAL — run if ./start.sh fails with port error
# Kill any process using ports 8080 and 5567
lsof -ti :8080 | xargs kill -9
lsof -ti :5567 | xargs kill -9
```

7.3 Start the server

```
TERMINAL — always use ./start.sh
./start.sh

# Expected output:
# Running jaspr in server rendering mode.
# [CLI] Web compilers started.
# [CLI] Done building web assets.
# [CLI] Server started.
# [SERVER] Serving at http://localhost:8080
```

7.4 Verify the full flow in browser

7. Open http://localhost:8080 | **verify Brew Guide home loads**
8. Open DevTools > Elements | **verify real HTML: section, h1, div.method-grid**
9. Click a card | **verify loading animation appears**
10. Wait for Gemini | **verify** coffee detail **page loads with all fields**
11. Open DevTools on detail page | **verify article, h1, dl, ul in DOM**
12. Click ☕ Brew Guide | **verify back navigation to home works**
13. Go to http://localhost:8080/?error=1 | **verify you have error banner** to explain the user

8. Phase 7 — Git commits

Commit after each working milestone. This creates restore points if something breaks.

TERMINAL — git commit sequence

```
# Initialize repo (run once)
git init
git add .
git commit -m "feat: initial Jaspr scaffolding"

# After theme and palette:
git add .
git commit -m "feat: SCA color palette and typography"

# After MethodSelector working:
git add .
git commit -m "feat: MethodSelector component with SCA palette"

# After Gemini integration working end-to-end:
git add .
git commit -m "feat: Gemini service and server middleware"

# After CoffeeDetail working:
git add .
git commit -m "feat: CoffeeDetail page with Gemini integration"

# After loading overlay and error handling:
git add .
git commit -m "feat: loading overlay, error recovery, back button fix"

# Final commit:
git add .
git commit -m "feat: complete Brew Guide demo ready for Flutter Conf"
```

9. README.md — update before the conference

The scaffolded README.md is generic. Replace it with project-specific content that the audience can find on GitHub.

README.md — replace scaffolded version

```
# Brew Guide

A specialty coffee web app built with Jaspr, Gemini, and Antigravity.
Demonstrated at Flutter Conf Orlando, July 16, 2026.

## Stack
- Jaspr 0.23.1 — Dart web framework with SSR
- Gemini API (gemini-3.5-flash) — AI coffee recommendations
- Antigravity CLI 1.0.5 (agy) — Spec-Driven Development agent

## Run locally

1. Clone the repo
2. Run: dart pub get
3. Get a Gemini API key from aistudio.google.com (billing required)
4. Create start.sh:
   #!/bin/bash
   export GEMINI_API_KEY="your_key_here"
   jaspr serve
5. Run: chmod +x start.sh && ./start.sh
6. Open: http://localhost:8080

## Talk
"Building Modern Web Apps with Jaspr, Gemini and Antigravity"
Flutter Conf Orlando — July 16, 2026
```

10. July 16 — pre-stage checklist

- Terminal: run ./start.sh and verify localhost:8080 loads
- Browser: open DevTools Elements tab — have it ready to show
- Antigravity IDE: open with project loaded
- IDE: grant all agy permissions (find, read, write)
- IDE: open lib/specs/gemini_service.md in editor
- Browser: test one full flow — card click → loading → detail → back → home
- Browser: test /?error=1 — verify error banner appears and disappears on click
- Browser: zoom to 125% for back-row visibility
- Notifications: disable all system notifications
- Screen: set to Do Not Disturb mode
- Backup: know the GitHub URL by memory in case someone asks

DEMO DAY FALLBACK

If Gemini fails during the demo: the error banner appears. Click the same card again. Say: "LLMs are non-deterministic. This is exactly why we write defensive error handling." Then continue normally.